

Podstawy Informatyki

INDIANA SNAKE

Aleksandra Woźniak, Justyna Sielska, Marta Lipień

INDIANA SNAKE W PIGUŁCE

- Zamiast standardowych owoców, gracz poluje na rozmaite **artefakty**, o pięciu różnych teksturach. Każdy zebrany przedmiot dodaje 1 punkt do wyniku, ale sprawia też, że Indiana Snake wydłuża się o 2 segmenty. Przez to z każdym ruchem robi się coraz ciaśniej i trudniej.
- Gra kończy się w momencie uderzenia w ścianę lub gdy wąż zaplącze się we własny ogon. Wymaga to od gracza planowania trasy z wyprzedzeniem, zwłaszcza, że artefakty nie zawsze pojawiają się w wygodnych miejscach.
- W grze poruszamy się strzałkami. Odpowiednie fragmenty kodu chronią użytkownika przed "cofnięciem się w ciało" snake'a.
- Gra posiada licznik punktów, który widać w rogu ekranu przez całą grę, który na bieżąco się aktualizuje.
- Po przegranej nie musisz resetować całej gry. W menu możesz wybrać opcję restaru lub wpisać się do leaderboardu. Możesz też wyjść do menu.

GENEROWANIE ARTEFAKTÓW

Tworzy losowe artefakty na mapie.

Każdy artefakt ma rozmiar 48x48.

Funkcja jest wywoływana przy starcie i
restarcie gry.

```
void artefakty(vector<SDL_Rect>& artifacts)
{
    artifacts.clear();

    artifacts.push_back({
        rand() % (WINDOW_WIDTH - 32), // Losowy X w granicach okna
        rand() % (WINDOW_HEIGHT - 32), // Losowy Y w granicach okna
        48, 48
    });
    textRand = (rand() % 5) + 1;
}
```

RUCH WĘŻA

Gracz steruje wężem strzałkami

Program blokuje zawracanie o 180 stopni

Dzięki temu wąż nie może „wejść w siebie” przez błąd gracza

Głowa węża przesuwa się o 20 pikseli

Ruch odbywa się w stałych odległościach więc gra jest płynna

```
// Blokada zawracania o 180 stopni
if (e.key.keysym.sym == SDLK_DOWN && dir != UP) { dir = DOWN; }
else if (e.key.keysym.sym == SDLK_UP && dir != DOWN) { dir = UP; }
else if (e.key.keysym.sym == SDLK_RIGHT && dir != LEFT) { dir = RIGHT; }
else if (e.key.keysym.sym == SDLK_LEFT && dir != RIGHT) { dir = LEFT; }
```

```
if (current_state == PLAYING) {

    switch (dir)
    {
        case DOWN:  head.y += 32; break;
        case UP:    head.y -= 32; break;
        case LEFT:  head.x -= 32; break;
        case RIGHT: head.x += 32; break;
    }
}
```

PRZECHOWYWANIE CIAŁA WĘŻA

Deque to struktura, która pozwala dodać nową głowę na początek i usunąć ogon z końca.

```
deque<SDL_Rect> rq; //cialo weza
```

- Nowa głowa jest dodana na przód
- Ogon jest usuwany tylko jeśli wąż jest za długi
- Wąż rośnie gdy zje artefakt

```
rq.push_front(head);  
  
while (rq.size() > (size_t)snake_size)  
{  
    rq.pop_back();  
}
```

KOLIZACJA

Kolizacja to stan w którym dwa obiekty na siebie nachodzą.

Kolizacja z artefaktem:

```
for (auto& artif : artifacts) {  
    if (SDL_HasIntersection(&head, &artif)) //jesli sie pokryje z artefaktem  
    {  
        Mix_PlayChannel(-1, eatSound, 0);  
        snake_size += 2; //jak je artefakt to rośnie o 2  
        //losujemy nowa lokalizacje artefaktu  
        artif.x = rand() % (WINDOW_WIDTH - 48);  
        artif.y = rand() % (WINDOW_HEIGHT - 48);  
        score++;  
        textRand = (rand() % 5) + 1;  
    }  
};
```

Wąż zjada artefakty, artefakt znika, wąż rośnie i gracz dostaje punkt.

Kolizacja węża z samym sobą:

```
//Sam ze soba - zakoncz gre  
for_each(rq.begin(), rq.end(), [&](auto& snake_segment) {  
    if (head.x == snake_segment.x && head.y == snake_segment.y)  
    {  
        Mix_HaltMusic();  
        Mix_PlayMusic(menubgm, -1);  
        current_state = GAMEOVER;  
    }  
});
```

- Jeśli głowa dotknęła któregoś segmenty to gra przechodzi do stanu Game Over.

Kolizacja ze ścianą:

```
//Ze sciana - zakoncz gre  
if (head.x < 0 || head.x > WINDOW_WIDTH - 32 || head.y < 0 || head.y > WINDOW_HEIGHT - 32)  
{  
    Mix_HaltMusic();  
    Mix_PlayMusic(menubgm, -1);  
    current_state = GAMEOVER;  
}
```

- Jeśli wąż próbuje wyjść poza granice okna gra się kończy.

MENU

Menu obsługiwane strzałkami.

Aktualnie wybrana opcja jest podświetlana.

Zatwierdzenie wyboru klawiszem Enter.

```
if(e.key.keysym.sym == SDLK_UP)
{
    chosenIndex = (chosenIndex - 1 + OPTIONS_NUM) % OPTIONS_NUM;
}
else if (e.key.keysym.sym == SDLK_DOWN)
{
    chosenIndex = (chosenIndex + 1) % OPTIONS_NUM;
}
```

```
//OBSŁUGA MENU INTERAKTYWNYCH
int chosenIndex = 0;
const int OPTIONS_NUM = 4;
```

LEADERBOARD

Każdy rekord składa się z nazwy gracza i wyniku, a dane przechowywane są w `vector<Player>`

```
struct Player
{
    string username;
    int HighScore;
};
```

- Dane są wczytywane z pliku do wektora poprzez customowo napisane do tego funkcje.
- Zachowane jest tylko 10 najlepszych wyników - pozostałe są skracane, a plik jest nadpisywany nową tabelą.

```
if (scores.size() > 10)
{
    scores.resize(10);
}

saveScores(scores);
```

- Sortowanie odbywa się malejąco po wyniku.

```
sort(scores.begin(), scores.end(), [](const Player& a, const Player& b) {
    return a.HighScore > b.HighScore;
});
```


WPISYWANIE NAZWY GRACZA

```
SDL_StartTextInput(); //aktywacja pisania
```

```
if (e.type == SDL_TEXTINPUT) //przyjmuje tekst i dodaje do inputa
{
    usernameinput += e.text.text;
}
```

Podczas włączonego trybu wpisywania tekstu program odbiera znaki jeden po drugim i dodaje je do nazwy gracza.

Wpisywany znak jest na bieżąco wyświetlany na ekranie.

```
if (e.key.keysym.sym == SDLK_BACKSPACE && usernameinput.length() > 0)
{
    usernameinput.pop_back();
}
```

- Klawisz Backspace pozwala usunąć ostatni znak.

DŹWIĘK I MUZYKA

W grze muzyka tła jest odtwarzana osobno w menu oraz podczas rozgrywki.

Przy zjedaniu artefaktu odtwarzany jest krótki efekt dźwiękowy, który informuje gracza o zdobyciu punktu.

Po zakończeniu gry muzyka zostaje zastąpiona muzyką menu.

```
Mix_PlayMusic(menubgm, -1);
```

```
Mix_PlayChannel(-1, eatSound, 0);
```

```
Mix_Music* gamebgm = Mix_LoadMUS("snake_main_music.mp3");  
Mix_Music* menubgm = Mix_LoadMUS("snake_intro_theme.mp3");  
Mix_Chunk* eatSound = Mix_LoadWAV("bite.ogg");
```

KONTROLA PRZEBIEGU PROGRAMU

Dzięki temu np. nie wąż nie porusza się po przegranej oraz w odpowiednim momencie wyświetla się Game Over.

Stany gry określają, czy gra jest aktualnie w trakcie rozgrywki, czy po przegranej oraz decydują o tym co jest wyświetlane i jak program reaguje na klawiaturę.

```
enum GameState //stany gry
{
    MENU,
    LEADERBOARD,
    PLAYING,
    GAMEOVER,
    SAVESCORE,
    CUSTOMIZE
};
```

ZRZUTY EKRANU GRY

TOP 10 GRACZY

1. LADYGAGA - 63
2. MIKIMUR - 54
3. GRZECHU - 50
4. JUSTYNKA - 47
5. BRUNO - 43
6. MIKIK - 40
7. MAŁEZYMIER - 37
8. BOBEREK - 28
9. MIKIOP - 27
10. LUSIA - 1

AKY POWROD

KOLOR GŁOWY:



KOLOR CIAŁA:



CIĘ DO MENU UŻYJ 'M'

GAME OVER

TWOJ WYNIK TO 1.
ROZPOCZNIJ DO NOWA
ZAPISZ SWOJ WYNIK
POWROD DO MENU

PODZIAŁ OBOWIĄZKÓW W ZESPOLE

- **Justyna Sielska:** Poprawki w kodzie, Mechanika artefaktów, podstawowa mechanika Game Over (wyłączanie gry oraz restart spacja) i ustalenie stanów gry, wyświetlanie wyniku w trakcie gry, brak Game Over przy "cofaniu", przygotowanie sprawozdania z projektu;
- **Marta Lipień:** Podstawowe ciało gry, czuwanie nad fabułą i assetami, przygotowanie modułu customowego koloru ciała oraz głowy snake'a + menu, przygotowanie prezentacji multimedialnej projektu;
- **Aleksandra Woźniak:** Obsługa błędów, udźwiękowienie programu, praca nad Leaderboardem - wyniki, zapis; Stworzenie ekranu Game Over z podanymi wynikami.

The background of the image is a stylized, abstract representation of a forest. It features dark, silhouetted tree trunks and foliage in the foreground, with lighter, teal-colored vertical bands representing the forest floor or distant trees. A white rectangular box with a thin black border is centered horizontally and vertically, containing the text "POKAZ GAMEPLAYU".

POKAZ GAMEPLAYU